

UNIVERSITÀ DEGLI STUDI DI MILANO-BICOCCA
DIPARTIMENTO DI INFORMATICA, SISTEMISTICA E COMUNICAZIONE
CORSO DI LAUREA IN INFORMATICA



Hallucinations degli LLM: come ridurle integrando un RAG basato su Knowledge Graph

Relatore: Dr. Giuseppe Vizzari

Co-relatore: Dr. ssa Blerina Spahiu

Relazione della prova finale di:

MONDONI ANDREA

879253

Anno Accademico 2023 - 2024

Indice

Elenco delle figure	iv
Elenco delle tabelle	v
List of acronyms	vi
1 Introduzione	1
2 HALLUCINATIONS	3
2.1 Cos'è un Hallucination	3
2.2 Esempi di Hallucinations	4
2.2.1 Llama vs ChatGPT	6
2.3 Possibili soluzioni alle Hallucinations	7
3 RETRIEVAL-AUGMENTED GENERATION (RAG)	10
3.1 Cos'è un RAG	10
3.2 Le fasi di un RAG	10
3.2.1 Embedding	12
3.2.2 Similarity function	13
3.3 Vantaggi RAG	13
3.4 Evoluzione dei RAG	14
3.5 KG-RAG	16
3.5.1 Cos'è un KG	16
3.5.2 Vantaggi di un KG-RAG	16
3.5.3 Struttura di un KG-RAG	16
4 IL MIO PROGETTO	20
4.1 Tecnologie/Metodologie	20
4.2 Preparazione architettura	21

4.2.1	Struttura KG	21
4.2.2	Calcolo degli Embeddings	24
4.3	Pipeline	25
4.4	Text2Cypher	27
4.5	Valutazione	29
4.6	Confronto Risultati	30
5	Conclusioni	33
6	Codice	34
6.1	Calcolo Embeddings	34
6.2	Pipeline	36
	Bibliografia	40

Elenco delle figure

2.1	Risposta corretta di ChatGPT-4 dei biscotti	4
2.2	Risposta errata di Llama2 dei biscotti	5
2.3	Risposta errata di Llama3 dei biscotti	5
2.4	Risposta corretta di ChatGPT-4 delle coordinate	5
2.5	Risposta errata di Llama2 delle coordinate	6
2.6	Risposta errata di Llama3 delle coordinate	6
2.7	Risposta errata di Llama2 su Carlo Magno	6
2.8	Risposta errata di Llama2 su Tupac Shakur	6
2.9	Confronto tra ChatGPT-4, Llama2 e Bard [9]	7
2.10	Esempio di prompt con tecnica Few-Shot [1]	8
2.11	Esempio di prompt senza Chain-of-Thought (CoT) vs con, in evidenza il ragionamento [16]	9
3.1	Pipeline di un RAG [10]	12
3.2	Rappresentazione 3D di varie parole [3]	12
3.3	Naive, advanced e modular RAG a confronto [4]	15
3.4	Pipeline CoE [12]	17
3.5	Esempio CoE [12]	18
4.1	Struttura PrimeKG [2]	21
4.2	Esempio funzionamento con embeddings	26
4.3	Esempio funzionamento Text2cypher	28
4.4	Esempio funzionamento Text2cypher con FewShot	29
4.5	Esempio funzionamento senza RAG	32
4.6	Esempio funzionamento con RAG	32

Elenco delle tabelle

3.1	Confronto tra Embedding-RAG e KG-RAG	18
3.2	Confronto tra EBR e KG-RAG.	19
4.1	Esempio informazioni di un nodo	22
4.2	Dettagli sul nodo "Medrysone" di tipo "drug" (<i>per questioni di spazio alcune informazioni sono state accorciate</i>)	23
4.3	Dettagli sul nodo "Hepatitis B Virus Infection" di tipo "disease" (<i>per questioni di spazio alcune informazioni sono state accorciate</i>)	24

Lista degli Acronimi

AI intelligenza artificiale

LLM Large Language Model

LM Language Model

RAG Retrieval-Augmented Generation

CoT Chain-of-Thought

NLP Natural Language Processing

RNN Recurrent Neural Networks

CNN Convolutional Neural Networks

CoE Chain of Explorations

PrimeKG Precision Medicine Knowledge Graph

EBR Entity-Based Retrieval

KG Knowledge Graph

KG-RAG Retrieval-Augmented Generation con Knowledge Graph

1

Introduzione

Negli ultimi anni, l'*intelligenza artificiale (AI)*, in particolare quella generativa, ha conosciuto uno sviluppo esponenziale entrando a far parte prepotentemente della nostra vita quotidiana, e si prevede che in futuro possa rivoluzionare completamente la nostra società. Nonostante siano diversi anni che vengono fatti continuamente passi avanti in questo ambito, l'AI è diventata accessibile a tutti con il rilascio, a fine 2022, di ChatGPT di OpenAI che nel giro di soli 2 mesi ha raggiunto 100 milioni di utenti, un dato impressionante se paragonato al tempo impiegato da alcune delle applicazioni più utilizzate oggi quali TikTok e Instagram, che hanno impiegato rispettivamente 9 e 30 mesi per raggiungere un bacino così ampio di utenti.

Nel nome ChatGPT, **GPT** è l'acronimo di *Generative Pre-trained Transformer*, ovvero l'algoritmo utilizzato per elaborare un testo scritto in linguaggio naturale al fine di generarne uno coerente, contestualmente rilevante e che simuli la scrittura umana.

GPT è un tipo di *Large Language Model (LLM)*, ovvero un modello di **deep learning** addestrato su un'enorme quantità di dati al fine di acquisire la conoscenza della sintassi, della semantica e della relazione tra le parole; questo permette di poter svolgere vari compiti quali ad esempio:

- Rispondere alle domande;
- Riassumere dei documenti;
- Tradurre in diverse lingue;

- Generare codice in diversi linguaggi di programmazione;
- Generare e classificare dei testi;
- Molto altro.

Questi sono solo alcune delle enormi potenzialità di un LLM, ma non bisogna dimenticare anche delle varie problematiche:

- I dati non sono recenti: le informazioni disponibili risalgono al periodo di addestramento del modello, il che lo rende incapace di rispondere a domande di attualità;
- La possibilità di violare i diritti d'autore: spesso infatti non è possibile rintracciare le fonti con cui è stato addestrato il modello, questo ha sollevato varie preoccupazioni riguardo ai diritti d'autore;
- Problemi etici: gli LLM possono perpetuare pregiudizi linguistici, di genere, razziali e politici presenti nei dati di addestramento;
- Hallucinations: delle volte vengono generate affermazioni false o inventate.

In questo elaborato si analizza il problema delle **Hallucinations** e delle possibili tecniche per evitarle/arginarle ponendo maggior attenzione sui *Retrieval-Augmented Generation* (RAG).

2

HALLUCINATIONS

2.1 Cos'è un Hallucination

Gli LLM hanno modificato il nostro modo di lavorare, ma non sempre possiamo fidarci delle risposte che generano, infatti può capitare che delle volte l'output generato sia magari corretto da un punto di vista sintattico o grammaticale, ma contenga al suo interno delle informazioni non veritiere, false e delle volte fuori contesto. Questo fenomeno prende il nome di Hallucinations, una possibile definizione è: *“hallucination is typically referred to as a phenomenon in which the generated content appears nonsensical or unfaithful to the provided source content”* [6].

Le cause di questo fenomeno sono molteplici, tra le principali troviamo [15]:

- Training data incompleti o con **rumore** possono condurre ad errori nella comprensione da parte del modello, il quale potrebbe fraintendere le informazioni e generare risposte errate;
- Ambiguità nelle domande degli utenti: se una domanda è poco chiara o ambigua, il modello potrebbe interpretarla in modo errato e fornire risposte inappropriate;
- **Overfitting**: quando il modello si adatta troppo bene ai dati di addestramento, potrebbe avere difficoltà a generalizzare su dati mai visti, producendo risposte errate;
- Presenza di **bias** nel training data;

Possiamo identificare diversi tipi di hallucinations [19]:

1. **Input-conflicting hallucination**, si verifica quando l'output generato non risponde correttamente alla richiesta dell'utente, avviene soprattutto nei casi in cui l'input include sia il compito (task) che altri dati, questo può condurre alla generazione di una risposta incoerente con la domanda iniziale;
2. **Context-conflicting hallucination**, accade quando la risposta del modello si contraddice oppure presenta delle incongruenze, questo è comune soprattutto quando vengono generate risposte lunghe a causa della limitata capacità dei modelli di gestire la memoria a lungo termine;
3. **Fact-conflicting hallucination**, quando la risposta generata contiene degli errori fattuali, ad esempio su informazioni riguardanti eventi, persone o dati specifici.

2.2 Esempi di Hallucinations

In questa sezione sono stati sottoposti ai modelli ChatGPT-4, Llama2:7b e Llama3:8b, diversi quesiti per verificare se essi producono delle Hallucinations[8].

Innanzitutto è stato sottoposto ai 3 modelli un indovinello logico-matematico, nelle Fig. 2.1, 2.2, 2.3 possiamo vedere le risposte e notare come soltanto ChatGPT-4 abbia prodotto il risultato corretto.

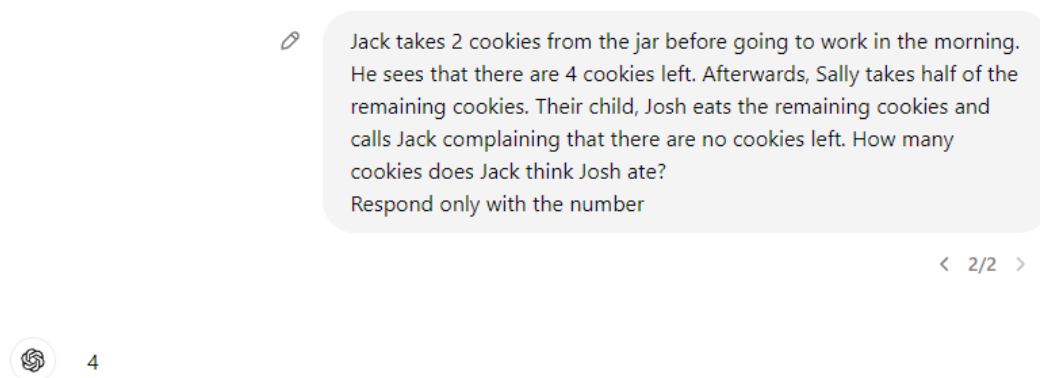


Figura 2.1 Risposta corretta di ChatGPT-4 dei biscotti

```
>>> Jack takes 2 cookies from the jar before going to work in the morning. He sees that there are 4 cookies left. Afterw
... ards, Sally takes half of the remaining cookies. Their child, Josh eats the remaining cookies and calls Jack complai
... ning that there are no cookies left. How many cookies does Jack think Josh ate?"

Jack thinks Josh ate 2 cookies.
```

Figura 2.2 Risposta errata di Llama2 dei biscotti

```
>>> Jack takes 2 cookies from the jar before going to work in the morning. He sees that there are 4 cookies left. Afterw
... ards, Sally takes half of the remaining cookies. Their child, Josh eats the remaining cookies and calls Jack complai
... ning that there are no cookies left. How many cookies does Jack think Josh ate?
Jack started with some number of cookies. Then he took away 2. There were 4 cookies left. So he had 6 to start
with. Sally then took half of those, which is 2. That means there are 2 left. Then Josh eats all the rest. He ate
2 - 1 = 1 cookie. The answer is 1.
```

Figura 2.3 Risposta errata di Llama3 dei biscotti

Dopodiché sono state sottoposte ai 3 modelli le coordinate geografiche di Milano, per verificare se essi sono in grado di riconoscerle, nelle Fig 2.4, 2.5, 2.6 possiamo vedere le risposte, anche in questo caso solo ChatGPT-4 ha risposto correttamente, infatti i modelli Llama hanno sbagliato la città, affermando che fosse Roma oppure Piacenza.

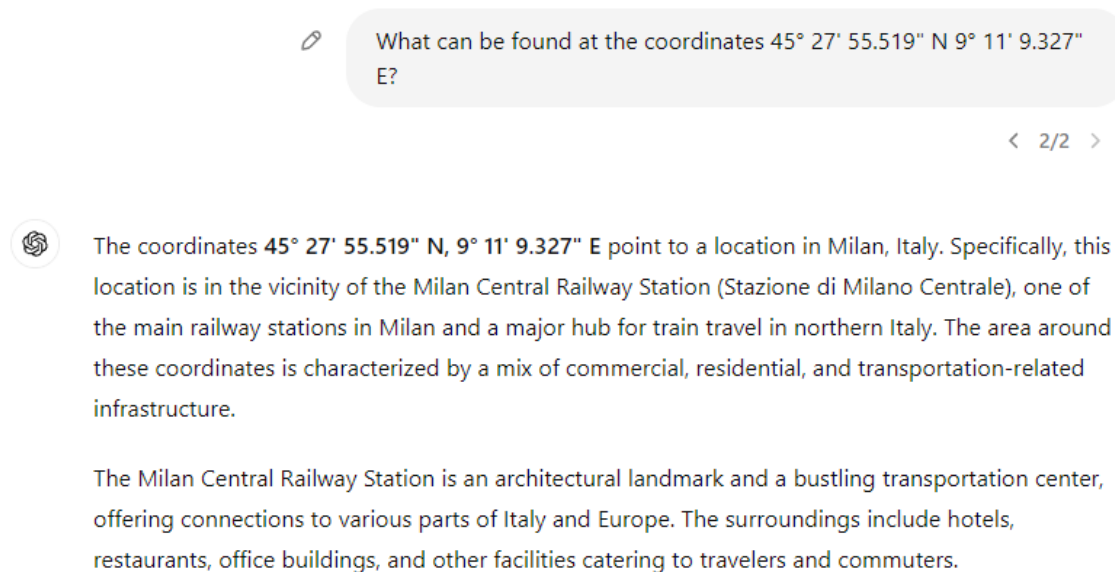


Figura 2.4 Risposta corretta di ChatGPT-4 delle coordinate

```
>>> What can be found at the coordinates 45° 27' 55.519" N 9° 11' 9.327" E?

The coordinates you provided are located in the city of Rome, Italy.

At these coordinates, you can find:

* The Spanish Steps (Scalinata di Spagna in Italian), a famous staircase in the heart of Rome.
* The Piazza del Popolo, a large public square that is a popular spot for tourists and locals alike.
* The Villa Borghese, a beautiful park with gardens, lakes, and walking paths.

These are just a few examples of what can be found at these coordinates. Let me know if you have any other questions!
```

Figura 2.5 Risposta errata di Llama2 delle coordinate

```
>>> What can be found at the coordinates 45° 27' 55.519" N 9° 11' 9.327" E?
A geographic mystery!

According to Google Maps, the coordinates 45° 27' 55.519" N 9° 11' 9.327" E correspond to a location in Italy, specifically in the town of Piacenza.

More specifically, this point is located at the city hall (Palazzo Comunale) of Piacenza, which is a medieval palace that serves as the seat of government for the city.
```

Figura 2.6 Risposta errata di Llama3 delle coordinate

Poi è stato chiesto quale evento storico importante si è verificato il 25 dicembre dell'800, la risposta corretta fornita da ChatGPT-4 e Llama3 è l'incoronazione di Carlo Magno, nella Fig. 2.7 è riportato l'errore di Llama2.

```
>>> What's the significant event that happened on december 25, 800?

There is no specific significant event that happened on December 25, 800 as this date does not exist in history. The year 800 does not correspond to any historical date or event.

Please let me know if you have any other questions
```

Figura 2.7 Risposta errata di Llama2 su Carlo Magno

Infine è stato domandato cosa accadde a Tupac Shakur nel marzo del 1997, ChatGPT-4 e Llama3 hanno risposto in maniera corretta dicendo che il rapper americano è morto nel '96, Llama 2 invece ha prodotto (Fig. 2.8) un testo contenente informazioni errate e contraddittorie.

```
>>> What happened to Tupac Shakur on March 9, 1997?

On March 9, 1997, American rapper Tupac Shakur was shot in Las Vegas, Nevada. He was driving with friends in a car when they were ambushed by a group of men in a black Cadillac. Tupac was hit by multiple bullets and died six days later on September 13, 1997, at the age of 25. The incident remains one of the most infamous unsolved crimes in hip-hop history
```

Figura 2.8 Risposta errata di Llama2 su Tupac Shakur

2.2.1 Llama vs ChatGPT

Dai precedenti esempi possiamo supporre che Llama2 è, tra quelli presi in considerazione, il modello più propenso nel generare Hallucinations, mentre ChatGPT-4 è il meno incline,

una possibile spiegazione a questo fenomeno potrebbe essere il numero di parametri: il modello di Meta è stato allenato con 7 miliardi, mentre quello di OpenAI, si suppone, con più di 1.5 mila miliardi. In diversi articoli questi modelli sono stati confrontati (Fig. 2.9), sottoponendoli a vari dataset di domande ed il risultato è che ChatGPT-4 surclassa in tutti i campi i modelli Llama2, specialmente la versione 7B che ottiene gli score peggiori; Llama3 si avvicina ai punteggi di ChatGPT-4 seppur ottenendoli leggermente inferiori. Possiamo quindi affermare che, a causa tra le altre cose, del numero molto limitato di parametri con cui è stato allenato, Llama2:7B è più propenso ad effettuare degli errori nella generazione del testo e a produrre Hallucinations.

Scores of Psychiatry National Licensing Examination

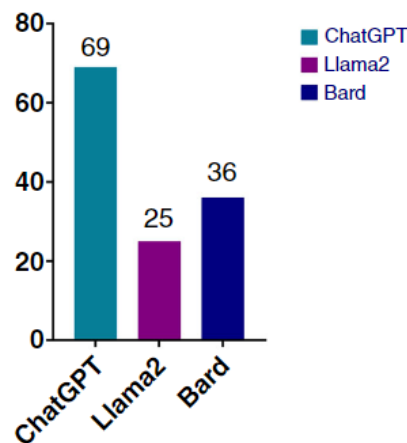


Figura 2.9 Confronto tra ChatGPT-4, Llama2 e Bard [9]

2.3 Possibili soluzioni alle Hallucinations

In base a quanto visto in precedenza, possiamo dire che le Hallucinations rappresentano un enorme problema degli LLM, ecco allora alcune possibili soluzioni sia da parte degli utenti sia dal lato dei programmatori:

- L'utente deve effettuare domande chiare, non ambigue e con il maggior contesto aggiuntivo possibile, ad esempio: *asking "Do cats talk?" could yield varied responses without clarification because cats indeed "talk" in cartoons and cats exhibit their own intricate system of vocalizations and body language in real life*[11];
- L'utente dovrebbe verificare le informazioni ottenute invece di fidarsi ciecamente;

- L'utente potrebbe eseguire più volte la stessa domanda per verificare se il modello fornisce risposte contrastanti;
- Migliorare il training data al fine di ridurre gli errori ed i bias in esso contenuti, questa è un'enorme sfida a causa dell'incredibile quantità di dati utilizzati per l'addestramento;
- **Prompt-engineering**, ovvero un insieme di tecniche che servono per manipolare il testo da fornire al modello al fine di renderlo più efficace, coerente ed accurato; eccone alcune[5]:
 - Utilizzare i **Few-Shot Prompts** anziché di **Zero-Shot**, ovvero invece di effettuare solo la domanda supportata da zero esempi, si forniscono al modello alcuni esempi contenenti l'output desiderato per guidarlo nella generazione della risposta corretta, in questo modo LLM acquisisce una maggiore consapevolezza delle intenzioni dell'utente. Questa tecnica aumenta le prestazioni del modello rispetto alla tecnica Zero-Shot, ma comporta un aumento nell'utilizzo dei **token** ed il rischio di raggiungere la lunghezza massima della finestra di input. In Fig. 2.11 è riportato un esempio di prompt dove si chiede al modello di classificare delle recensioni come "positive" o "negative" fornendogli degli esempi;

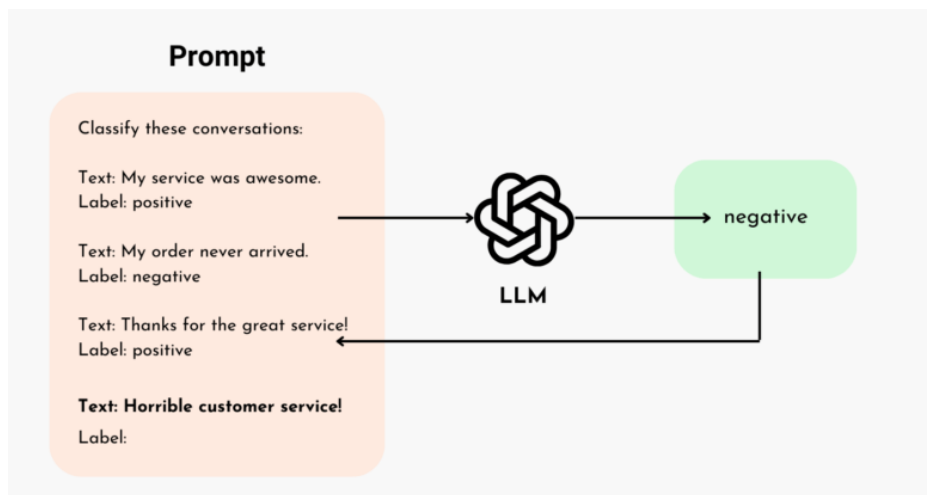


Figura 2.10 Esempio di prompt con tecnica Few-Shot [1]

- La **Chain-of-Thought** (CoT), è una tecnica nella quale si guida LLM nella risoluzione della task scomponendo una richiesta complessa in parti più piccole, questo aiuta il modello a risolvere una serie di passaggi intermedi anziché rispondere direttamente alla domanda. Questo metodo ha migliorato l'**accuracy** dal 17.7%

al 78.7% sul test MultiArith che comprende varie domande logico-matematiche [7], come quella riportata in Fig. 2.11. Si può notare come senza l'utilizzo di questa tecnica il modello genera una risposta errata, mentre grazie al CoT è in grado di replicare il ragionamento umano arrivando così al risultato corretto;

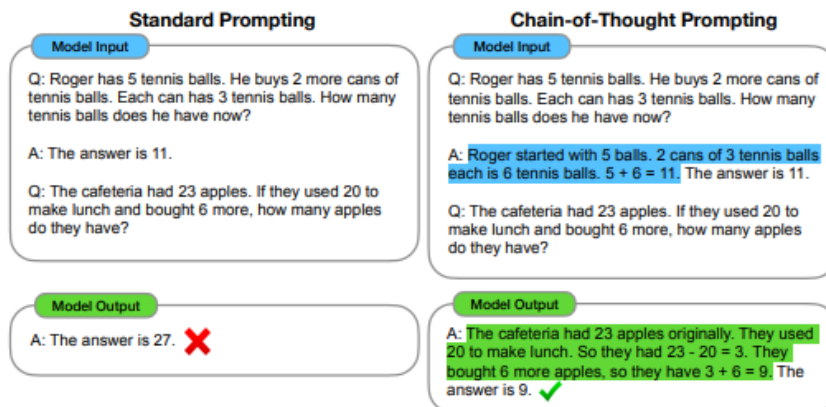


Figura 2.11 Esempio di prompt senza CoT vs con, in evidenza il ragionamento [16]

- **Fine-tuning**, è un metodo attraverso cui si prosegue l'addestramento di un LLM pre-addestrato aggiornandone i pesi, al fine di migliorare le prestazioni utilizzando un insieme di dati a cui non era stato ancora esposto. Questo processo permette di rendere il modello più specifico per un determinato dominio. Ad esempio un modello di linguaggio pre-addestrato su testi generici può essere sottoposto a fine-tuning per rispondere a domande in ambito medico, utilizzando un dataset di domande e risposte in medicina;
- **RAG**, combina le tecniche di recupero di informazioni con la generazione di testi, avvalendosi di un'ampia base di dati per produrre risposte pertinenti e corrette.

3

RETRIEVAL-AUGMENTED GENERATION (RAG)

3.1 Cos'è un RAG

Per mitigare il problema legato alla generazione delle Hallucinations, sono emersi come un ottimo strumento i *Retrieval-Augmented Generation (RAG)*, che consistono nel trovare ed incorporare delle informazioni rilevanti provenienti da un database esterno come contesto aggiuntivo da fornire in input al LLM. Questi dati estratti permettono al LLM di avere una conoscenza più approfondita, facilitando la generazione di risposte più accurate e pertinenti. La qualità del testo generato dal modello dipende quindi inevitabilmente dalle informazioni ricavate: si presenta quindi il problema di come effettuare questa operazione nella maniera più efficiente.

3.2 Le fasi di un RAG

1. L'**Indexing**, l'obiettivo di questa fase iniziale è la creazione del database contenente le informazioni esterne che verrà consultato in futuro dal LLM, è quindi fondamentale creare una fonte di conoscenza efficiente e di qualità.
Consiste innanzitutto nell'estrazione e nella pulizia dei dati dai vari documenti, anche in formati differenti (PDF, HTML, XLS, ...), successivamente queste informazioni

vengono suddivise in piccole parti dette **chunks**. La scelta della grandezza di questi è un punto focale, poiché dei chunk troppo grandi contengono più contesto ma al costo di generare del rumore, al contrario invece se questi sono troppo piccoli potrebbero non catturare bene il contesto. Per la creazione esistono vari criteri sia sintattici sia semantici, ad esempio si possono creare dei chunk formati da un numero definito di caratteri o token, questo metodo ha come conseguenza che spesso alcune parole oppure frasi vengono spezzate, rischiando così di perdere il filo logico del contesto. Un modo per risolvere questa situazione è ricorrere ad una divisione basata ad esempio sui paragrafi; si può inoltre utilizzare una separazione per argomenti che risulta essere più complicata ma più efficiente.

L'ultimo passo consiste nella trasformazione di ognuno di questi chunk in vettori numerici attraverso l'utilizzo di un modello di **embedding**, i vettori così ottenuti vengono salvati in un database vettoriale. Vedi 3.2.1;

2. Il **Retrieval**, la query dell'utente viene trasformata in un vettore numerico grazie allo stesso modello di embedding utilizzato in precedenza, a questo punto si effettua un confronto tra questo vettore e tutti quelli presenti nel database e si assegna ad ognuno uno **score** in base a delle politiche di similarità. Infine si selezionano in base al punteggio ottenuto i primi k-risultati che *dovrebbero* corrispondere al contesto più rilevante. Vedi 3.2.2;
3. L'**Augment**, la query dell'utente ed il contesto aggiuntivo trovato durante la fase di retrieval vengono inseriti nel prompt del LLM, bisogna fare attenzione a non superare il numero massimo di token che formano la finestra di input, inoltre fornire troppe informazioni potrebbe creare confusione nel modello, producendo così l'effetto contrario a quanto desiderato;
4. La **Generation**, a questo punto LLM può generare la risposta utilizzando sia le informazioni apprese durante la fase di addestramento sia quelle presenti nel contesto aggiuntivo fornitogli.

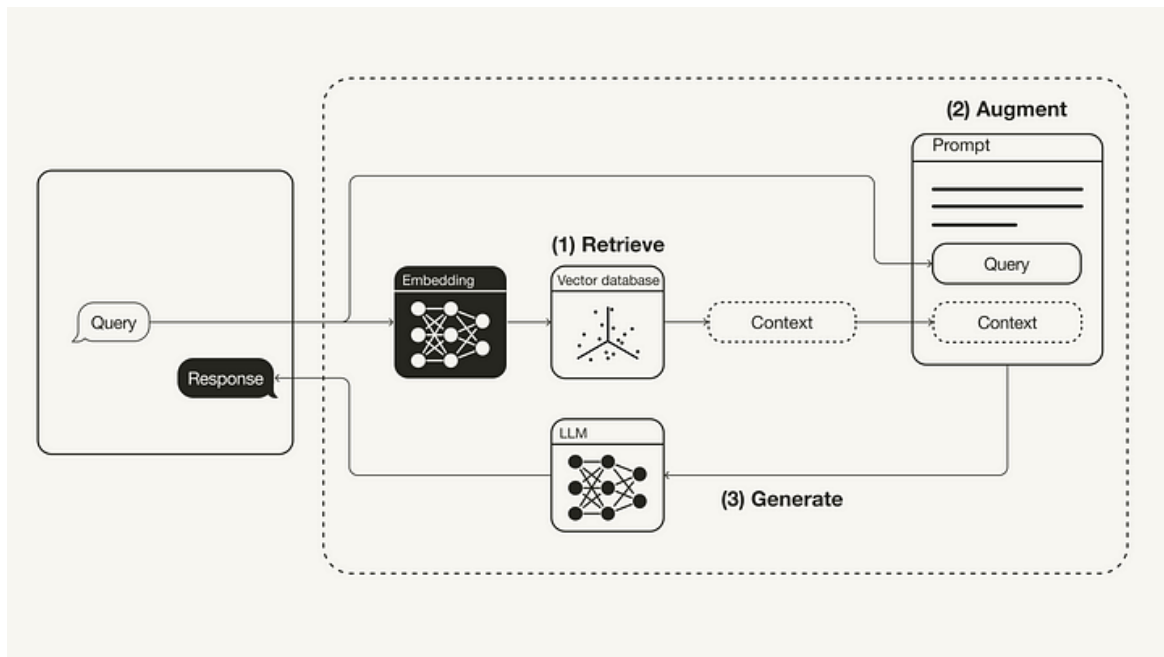


Figura 3.1 Pipeline di un RAG [10]

3.2.1 Embedding

L'embedding è una tecnica utilizzata nell'apprendimento automatico e nel *Natural Language Processing (NLP)* per rappresentare dei dati sotto forma di vettore numerico a n -dimensioni, così facendo è possibile rappresentare le informazioni in uno spazio vettoriale dove la vicinanza tra i punti indica una certa somiglianza. Nella Fig. 3.2 possiamo notare come la distanza tra le parole "man" e "woman" sia la stessa che tra "king" e "queen", questa rappresentazione permette al modello di comprendere che tra queste parole esiste una correlazione.

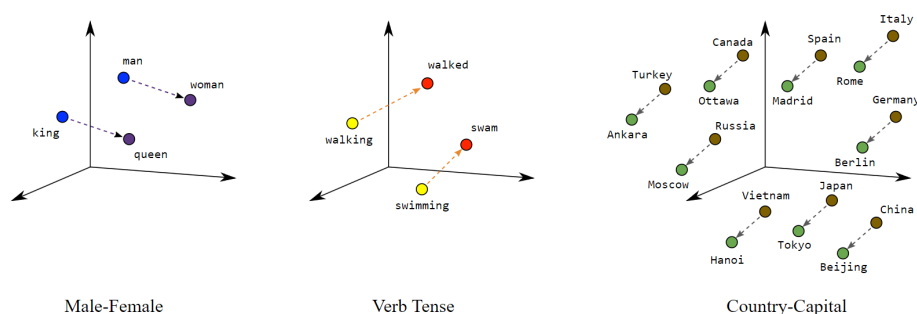


Figura 3.2 Rappresentazione 3D di varie parole [3]

È possibile effettuare l'embedding su diversi tipi di dati: dai più semplici come le parole grazie all'uso ad esempio di **Word2Vec**, ad informazioni più strutturate come frasi o paragrafi grazie a modelli più avanzati come **BERT**, fino ad informazioni più complesse come immagini ed audio attraverso l'uso rispettivamente di *Convolutional Neural Networks (CNN)* e di *Recurrent Neural Networks (RNN)*.

3.2.2 Similarity function

Le Similarity function sono un gruppo di funzioni matematiche utilizzate per calcolare la somiglianza di due oggetti, le più utilizzate sono:

- **Euclidean Similarity**, calcola la distanza che intercorre tra i due punti, restituendo sempre un numero reale non negativo che può variare da 0, quando i due punti coincidono, fino ad infinito nel caso di punti estremamente distanti.

$$\text{euclidean similarity} = \sqrt{\sum_{i=1}^n (A_i - B_i)^2}$$

Dove **A** e **B** sono i vettori, e A_i e B_i sono le i-esime componenti.

- **Cosine Similarity**, calcola il coseno dell'angolo formato dai due punti, dove il -1 indica che sono completamente diversi mentre 1 indica che sono la stessa entità.

$$\text{cosine similarity} = \cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \cdot \sqrt{\sum_{i=1}^n B_i^2}}$$

Dove **A** e **B** sono i vettori, e A_i e B_i sono le i-esime componenti.

La scelta tra l'utilizzo di queste formule per calcolare la somiglianza tra due vettori dipende molto dalle caratteristiche dei dati e dallo scopo finale: la Cosine similarity non tiene conto della **magnitudine** (lunghezza del vettore) e viene utilizzata per **NLP** come l'analisi di testi ed il confronto di documenti; la Euclidean Similarity si preferisce in quei casi dove si vuole valutare la reale distanza dei punti come ad esempio per il **clustering**[13].

3.3 Vantaggi RAG

- Uno dei punti forti dei RAG riguarda sicuramente la facilità della gestione dei dati: aggiungere nuove informazioni alla base di conoscenza risulta essere molto semplice

ed economico, è infatti solamente necessario calcolare l'embedding dei dati ed inserirlo nel **vetctor database**. In un classico LLM è invece necessario ri-addestrare l'intero modello, andando così incontro a notevoli spese. Proprio per questo i RAG possono contenere informazioni molto più recenti, andando così a sopperire l'obsolescenza dei dati e rendendo il modello in grado di rispondere a domande di attualità;

- Se durante la fase di Indexing sono stati inseriti in ogni chunk anche i **metadati** (come ad esempio l'autore, il titolo, la pagina in cui sono inserite quelle informazioni ecc), il LLM può fornire la fonte dei dati che cita nella sua risposta, in questo modo è possibile risalire al documento originale ed in caso in cui questo contenga degli errori può essere rapidamente corretto;
- Utilizzare un database vettoriale rende le operazioni di inserimento e di ricerca molto rapide;
- Grazie alla enorme possibilità di personalizzazione e customizzazione dovuta all'architettura RAG, le aziende possono implementare qualsiasi LLM e potenziarlo in modo che restituisca risultati rilevanti per la loro organizzazione;
- Questa tecnica è molto importante quando le aspettative degli utenti sulla pertinenza e sull'accuratezza delle risposte sono elevate, come accade nei **chatbot**;
- Garantiscono maggiore precisione e accuratezza delle risposte, riducendo così le Hallucinations;

3.4 Evoluzione dei RAG

Di seguito è riportata l'evoluzione dei RAG, dalla loro forma più basilare e semplice fino alla più complessa ed adattabile alle esigenze [4]:

1. **Naive RAG**, sono la prima versione di RAG nonché la più semplice, la loro struttura è quella descritta in precedenza (Vedi 3.2). Presentano diverse limitazioni quali la difficoltà nel retrieval delle informazioni per le query più complesse;
2. **Advanced RAG**, per superare i limiti di un naive RAG, vengono introdotti una fase di *pre-retrieval* e *post-retrieval*:
 - Durante il pre-retrieval, il sistema si focalizza sull'ottimizzazione della query dell'utente al fine di renderla più chiara per il modello attraverso meccanismi di *query rewriting*, *query transformation* e *query expansion*;

- L'obiettivo della fase di post-retrieval è integrare in maniera efficiente la query iniziale con il contesto ricavato, vengono quindi selezionate sole le informazioni essenziali, enfatizzando le sezioni più importanti e riducendo la lunghezza del contesto.
3. **Modular RAG**, progettato per essere molto più versatile ed adattabile rispetto alle precedenti architetture, è composto da una serie di moduli interconnessi, facilmente sostituibili, dove ciascuno è responsabile di un compito specifico nel processo di retrieval oppure di generazione. Sono presenti ad esempio retriever personalizzabili con meccanismi di recupero ibridi, che combinano la ricerca per parole chiave con quella semantica ed il recupero basato sull'apprendimento automatico, portando a informazioni più accurate e pertinenti. I retriever possono inoltre adattarsi a dati e requisiti in continua evoluzione, garantendo che le informazioni più rilevanti siano sempre accessibili. La natura *plug-and-play* dei moduli assicura che il sistema sia facilmente scalabile, gestendo volumi crescenti di dati e interazioni con gli utenti;

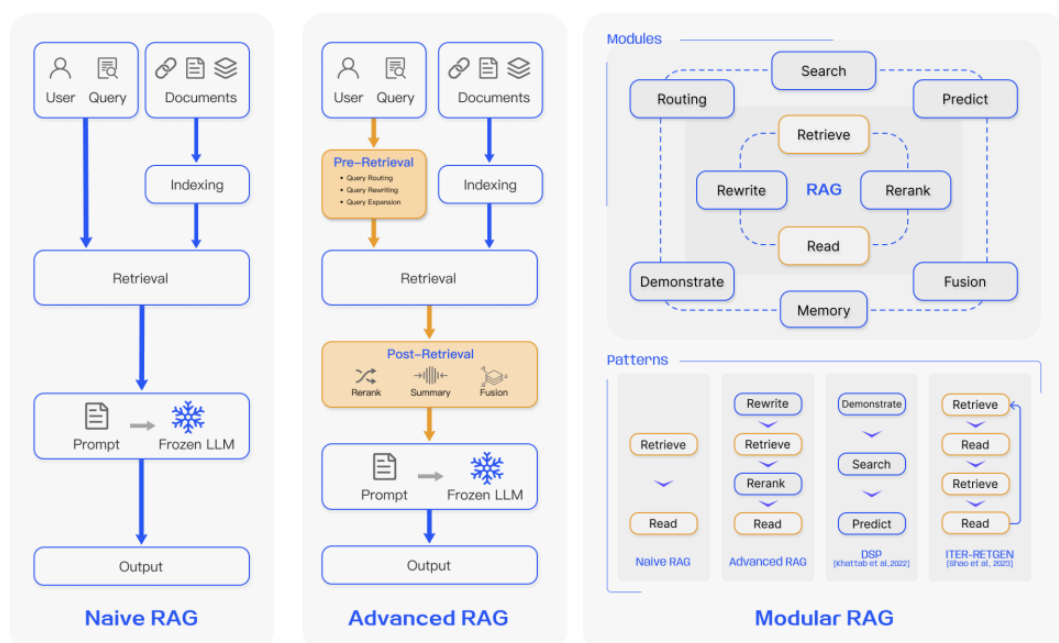


Figura 3.3 Naive, advanced e modular RAG a confronto [4]

3.5 KG-RAG

3.5.1 Cos'è un KG

Un *Knowledge Graph* (KG) è una rappresentazione grafica della conoscenza, dove i dati sono organizzati in **nodi**, rappresentazioni di entità e concetti, e **relazioni**, ovvero delle connessioni tra questi. È così possibile rappresentare in maniera strutturata delle informazioni complesse sotto forma di triple: $(entità) - [relazione] \rightarrow (entità)$. Ogni nodo ed ogni relazione possono contenere degli **attributi**.

3.5.2 Vantaggi di un KG-RAG

Il funzionamento dei RAG descritto in precedenza (Fig. 3.1) mostra delle limitazioni quando gli vengono poste delle query complicate, poiché il **retriever** potrebbe recuperare blocchi di testo che sono simili ma non esattamente pertinenti alla domanda, oppure fornire troppo contesto; *"An optimal RAG system would accurately retrieve only the necessary content, minimizing the inclusion of irrelevant information"* [12]. I **Modular RAG** presentano un'architettura complessa che richiede quindi un'attenta progettazione ed una notevole difficoltà nello sviluppo e nella manutenzione, un altro notevole svantaggio riguarda gli elevati costi computazionali legati a questo approccio. Una possibile alternativa consiste nell'integrare i RAG con i KG, questo comporta che la fase di retrieval viene effettuata direttamente sul grafo. I vantaggi di questo approccio sono diversi:

- Maggior comprensione delle connessioni tra concetti, in quanto le relazioni tra essi sono esplicite;
- Possibilità di **inferenza** sul grafo, scoprendo così delle relazioni implicite;
- I KG sono perfetti per rappresentare informazioni di domini quali quello medico, legale, ricerca ecc.

3.5.3 Struttura di un KG-RAG

Approccio 1

Di seguito è descritto l'approccio proposto per realizzare un *Retrieval-Augmented Generation con Knowledge Graph* (KG-RAG) del seguente paper [12]:

1. **KG Construction**, in questa fase iniziale avviene la costruzione del KG, attraverso l'estrazione di triple dai testi grazie all'utilizzo di un *Language Model (LM)* addestrato con la tecnica few-shot;
2. **Retrieval**, il grafo viene esplorato attraverso l'algoritmo *Chain of Explorations (CoE)* alla ricerca delle informazioni più rilevanti, questa è la parte cruciale di questo approccio (Vedi fig. 3.5). L'algoritmo è formato da tre parti:
 - (a) **Planning**, in questa fase si elabora una sequenza di passi che guideranno l'esplorazione del KG, vengono identificati i primi *top-k starting nodes* utilizzando una similarity function;
 - (b) **KG lookups**, vengono eseguite delle query **cypher** per trovare i nodi connessi e le relazioni con quelli di partenza, dopodiché viene utilizzato un LLM per filtrare i risultati selezionando i più rilevanti;
 - (c) **Evaluation**, il sistema decide se continuare l'esplorazione, modificando oppure no la pianificazione iniziale, oppure se passare alla fase successiva;
3. **Response Generation**.

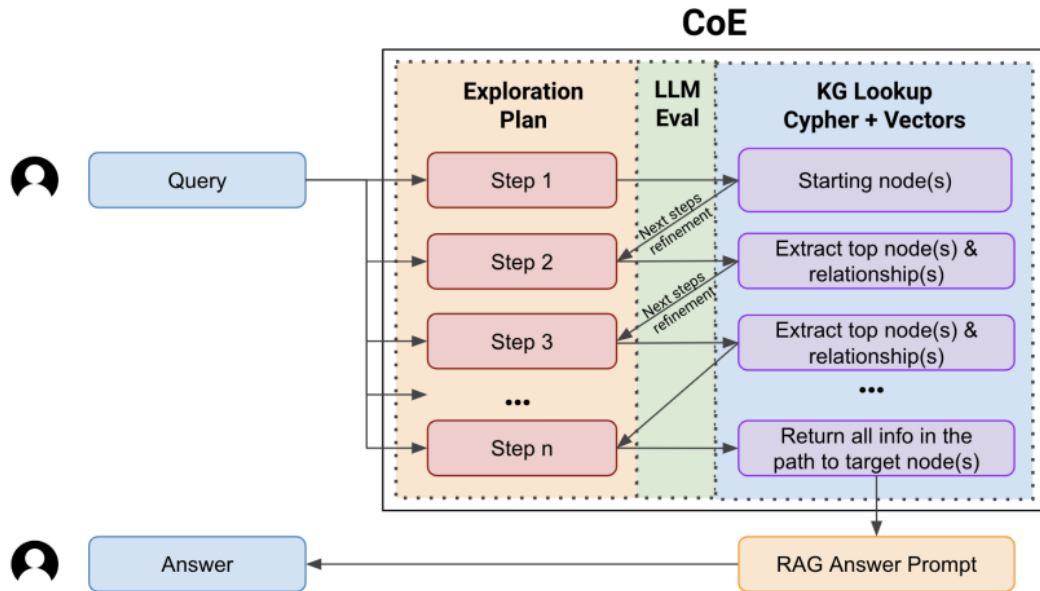


Figura 3.4 Pipeline CoE [12]

Questa pipeline è stata in seguito confrontata con un RAG basato su embedding, mettendole alla prova su vari dataset, in Tab. 3.1 sono mostrati risultati.

Model	EM	F1 Score	Accuracy	Hallucination
Human	63	-	-	-
Embedding-RAG	28	37	46	30
KG-RAG	19	25	32	15

Tabella 3.1 Confronto tra Embedding-RAG e KG-RAG

Possiamo notare come il KG-RAG ottenga dei risultati peggiori sia negli **Exact Match (EM)**, sia nell'**Accuracy** ed anche nel **F1 Score**, ad indicare come il modello presenti delle difficoltà nell'ottimizzazione della fase di retrieval; possiamo però vedere anche come riduca sensibilmente le Hallucinations che passano dal 30 al 15%. Nonostante le potenzialità, possiamo notare che questo approccio debba ancora migliorare sotto il punto di vista del recupero delle informazioni più adeguate.

```

Q: Which former husband of Elizabeth Taylor died in Chichester?
Step1: Start at target entity: "Elizabeth Taylor"
selected_nodes = ["Elizabeth Taylor", "Liz Taylor"]
Step2: List former husbands
selected_rels = ["married to", "married", "was married to"]
selected_nodes = ["Michael Wilding", "Conrad Nicky Hilton Jr", "Eddie Fisher"]
Step3: Identify places of death for each husband
selected_rels = ["died in"]
selected_nodes = ["hospital near his home in Sussex town of Chichester", "Chichester, West Sussex"]
Step4: Isolate the husband who died in Chichester
return (Elizabeth Taylor)-[married]→(Michael Wilding)-[died in]→(Chichester, West Sussex)
Answer Generated: Michael Wilding

```

Figura 3.5 Esempio CoE [12]

Approccio 2

In quest'altro approccio l'obiettivo è creare un chatbot di Customer Service basato su dei ticket già risolti, per farlo è stato implementato un KG-RAG [17]:

1. Innanzitutto avviene la costruzione del KG, dove ogni ticket viene rappresentato come un **albero**, nel quale ogni sezione del ticket è un nodo (definita come relazione *Intra-issue*). Dopodiché vengono realizzate le connessioni *Inter-issue* tra ticket differenti;
2. Per facilitare la fase di retrieval vengono calcolati gli embeddings di ogni nodo;
3. Durante la fase di retrieval, la query dell'utente viene analizzata per estrarre **coppie chiave-valore**, dove le chiavi sono gli elementi del grafo ed i valori sono le informa-

zioni estratte dalla query. In seguito si utilizza la cosine similarity per trovare i nodi più rilevanti, gli score così ottenuti dai nodi di uno stesso albero vengono sommati e vengono così restituiti i *top-k ticket*;

4. **Subgraph extraction**, viene utilizzata una query cypher contenete i *top-k ticket* per estrarre la porzione del KG più rilevante.
5. infine avviene l'**Answer Generation**.

Nella Tab. 3.2 è riportato il confronto effettuato tra questo approccio rispetto ad un classico *Entity-Based Retrieval (EBR)*, utilizzando come metriche *BLEU*, *METEOR* e *ROUGE*; tutte e tre misurano, seppur ciascuna con parametri differenti, quanto il testo generato dal modello è allineato e coerente con le risposte corrette del dataset di riferimento. Dai risultati ottenuti dal KG-RAG si può notare come surclassi completamente la controparte.

Model	BLEU	METEOR	ROUGE
EBR	0.057	0.279	0.183
KG-RAG	0.377	0.613	0.546

Tabella 3.2 Confronto tra EBR e KG-RAG.

4

IL MIO PROGETTO

In questo capitolo si approfondisce lo sviluppo di un chatbot creato per rispondere a vari quesiti medici, per farlo si utilizza un RAG basato su KG.

4.1 Tecnologie/Metodologie

Prima di approfondire la realizzazione, è importante introdurre le tecnologie utilizzate:

- Per la costruzione e la gestione del KG è stato impiegato **Neo4j**, un *Graph database* interrogabile attraverso query cypher;
- Per generare gli embedding si è optato per **all-MiniLM-L6-v2**, un *sentence-transformers model* che mappa frasi o paragrafi in un vettore a 384 dimensioni;
- Come LLM è stato scelto **Llama3**, ultima versione dei modelli di *Meta* rilasciato nel aprile del 2024 nelle versioni 8B e 70B, è stata scelta quella da 8 miliardi di parametri. Inoltre la **temperatura** è stata diminuita dallo 0.5 di default a 0.

4.2 Preparazione architettura

4.2.1 Struttura KG

Come già evidenziato in precedenza la costruzione del KG è una tappa fondamentale nella costruzione di un RAG in quanto questo determinerà la qualità delle informazioni recuperate, per questo motivo è stato scelto *Precision Medicine Knowledge Graph (PrimeKG)* come grafo. Questo contiene oltre **100 mila nodi** e più di **4 milioni di relazioni**, con dati estratti da 20 dei migliori dataset medici. In Fig. 4.1 è rappresentata la struttura del grafo, è possibile notare i diversi tipi di entità e le possibili relazioni tra essi.

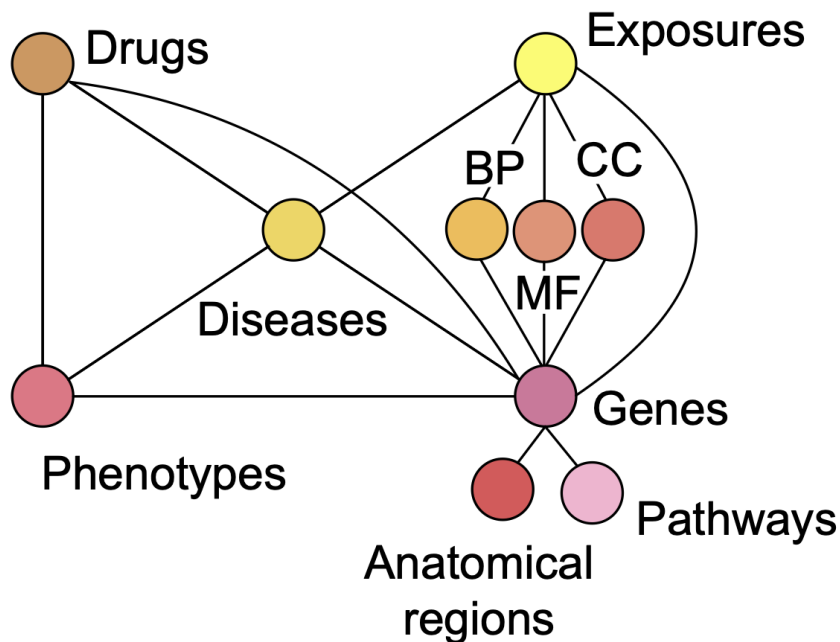


Figura 4.1 Struttura PrimeKG [2]

Innanzitutto sono stati importati i file *nodes.tab* ed *edges.csv* (link) che rappresentano rispettivamente i nodi e le relazioni. In Tab. 4.1 è riportato un nodo d'esempio con i rispettivi attributi che includono l'identificatore, il nome, il dataset di provenienza ed infine il tipo. L'unico attributo presente invece nelle relazioni indica solamente il tipo dei due vertici che collega.

Attributo	Valore
node_index	5297
node_name	PI4KA
node_source	NCBI
node_type	gene/protein

Tabella 4.1 Esempio informazioni di un nodo

Vista la scarsità di dati a disposizione, sono stati importati anche i file *disease-feature.tab* e *drug-feature.tab* che contengono varie informazioni, ad esempio una spiegazione della malattia/medicina, possibili sintomi, complicazioni ecc...

In Tab. 4.2 e 4.3 è possibile vedere un esempio di un nodo *drug* ed uno *disease*.

Attributo	Valore
node_index	14016
atc_1	Medrysone is anatomically related to sensory organs.
atc_2	Medrysone is in the therapeutic group of ophthalmologicals.
atc_3	Medrysone is pharmacologically related to antiinflammatory agents.
atc_4	The chemical and functional group of Medrysone is corticosteroids, plain.
category	Medrysone is part of Adrenal Cortex Hormones; Anti-Inflammatory Agents; Corticosteroid Hormone Receptor Agonists; Corticosteroids; Cytochrome P-45.
clogp	The log p value of Medrysone is 3.36.
description	Medrysone is a corticosteroid used in ophthalmology.
group	Medrysone is approved.
indication	For the treatment of allergic conjunctivitis, vernal conjunctivitis, episcleritis, and epinephrine sensitivity.
mechanism_of_action	There is no generally accepted explanation for the mechanism of action of ocular corticosteroids.
molecular_weight	The molecular weight of Medrysone is 344.5.
node_name	Medrysone
node_source	DrugBank
node_type	drug
pharmacodynamics	Medrysone is a topical anti-inflammatory corticosteroid for ophthalmic use.
state	Medrysone is a solid.
tpsa	Medrysone has a topological polar surface area of 54.37.

Tabella 4.2 Dettagli sul nodo "Medrysone" di tipo "drug" (per questioni di spazio alcune informazioni sono state accorciate)

Attributo	Valore
node_index	37752
mayo_causes	Hepatitis B infection is caused by the hepatitis B virus. The virus is passed from person to person through blood, semen or other body fluids.
mayo_complications	Having a chronic HBV infection can lead to serious complications, such as: Scarring of the liver.
mayo_prevention	The hepatitis B vaccine is typically given as three or four injections over six months. You can't get hepatitis B from the vaccine.
mayo_risk_factors	Hepatitis B spreads through contact with blood, semen or other body fluids from an infected person.
mayo_see_doc	When to see a doctor: If you know you've been exposed to hepatitis B, contact your doctor immediately.
mayo_symptoms	Signs and symptoms of hepatitis B range from mild to severe.
mondo_definition	A viral infection caused by the hepatitis B virus.
mondo_name	hepatitis B virus infection
node_name	hepatitis B virus infection
node_source	MONDO
node_type	disease
umls_description	Your liver is the largest organ inside your body. It helps your body digest food, store energy, and remove poisons. Hepatitis is an inflammation of the liver.

Tabella 4.3 Dettagli sul nodo "Hepatitis B Virus Infection" di tipo "disease" (*per questioni di spazio alcune informazioni sono state accorciate*)

4.2.2 Calcolo degli Embeddings

Dopo aver importato correttamente il KG, si procede calcolando un vettore di embedding per ciascun nodo, inserendolo come attributo del nodo stesso (Vedi 6.1 per il codice): prima di tutto, si recuperano le entità del grafo con i rispettivi attributi, questi vengono concatenati in un unico testo che viene poi passato a *all-MiniLM-L6-v2*; a seguito del calcolo del vettore da

parte del modello, si esegue una query cypher per aggiornare i nodi aggiungendo l'embedding come attributo. Poiché gli attributi in questione contengono poco testo, si è deciso di non effettuare una divisione in chunk per evitare che frammenti troppo piccoli potessero non comprendere il contesto generale. Infine viene creato su Neo4j un **vector database** che servirà in futuro per estrarre le entità più rilevanti. Di seguito il comando per la creazione: è necessario indicare la dimensione dei vettori e la similarity function che si desidera utilizzare.

```
CREATE VECTOR INDEX `fullindex` IF NOT EXISTS
FOR (n:Node) ON (n.embedding)
OPTIONS {indexConfig: {
  `vector.dimensions`: 384,
  `vector.similarity_function`: 'cosine'
}};
```

4.3 Pipeline

A questo punto la struttura del chatbot è terminata e possiamo concentrarci sull'elaborare le richieste in input: innanzitutto la query dell'utente (Fig. 4.2) viene trasformata grazie a *all-MiniLM-L6-v2* in un vettore di embedding che viene passato alla seguente funzione, la quale interroga il vector database ed assegna ad ogni nodo uno score utilizzando la cosine similarity.

```
def query_context(question_embedding):
    # Query to find context based on question embedding
    nodes = kg.query("""
CALL db.index.vector.queryNodes(
  'fullindex',
  129375,
  $question_embedding
) YIELD node AS n, score
RETURN n.node_name AS name, n.category AS category, n.pharmacodynamics AS pharmacodynamics,
  n.group AS group, n.half_life AS half_life, n.indication AS indication,
  n.mechanism_of_action AS mechanism_of_action, n.protein_binding AS protein_binding,
  n.state AS state, n.node_index as node_index, n.node_type AS Type, n.node_source AS Source,
  n.mayo_causes AS mayo_causes,
  n.mayo_complications AS mayo_complications,
  n.mayo_risk_factors AS mayo_risk_factors,
  n.mayo_symptoms AS mayo_symptoms,
  n.mondo_definition AS mondo_definition, score
""", params={"question_embedding": question_embedding})
    return nodes
```


I nodi così ricavati, vengono riordinati e si selezionano solo i 3 con lo score maggiore che *dovrebbero* rappresentare il contesto più significativo per rispondere alla richiesta dell'utente, questi vengono quindi concatenati in un'unica stringa insieme alla query iniziale e viene tutto passato al modello customizzato di Llama3, il quale genera la risposta. In Fig. 4.2 si può vedere un esempio di query, il nome dei 3 nodi ricavati con il relativo score ed infine la risposta del modello. Di seguito il codice sopra-descritto:

```
def main():
    question = input("Inserisci la domanda: ")
    # Calculate the embedding
    question_embedding = calculate_embedding(question)
    context = query_context(question_embedding)
    context.sort(key=lambda obj: obj['score'], reverse=True)

    print("\nContesto:")
    for node in context[:3]:
        print(node['name'], node['score'])

    top_context = context[:3]
    # Concatenate context and question
    input_text = f"""TASK: You are an AI assistant. Respond in paragraph using only the context
    ↳ provided, giving more relevance to the one with the higher score.
    QUESTION: {question}
    CONTEXT: {top_context} """

    # Get the response from the model
    response = query_llama3_instruct(input_text)
    print("\nRisposta dal modello:")
    print(response)

if __name__ == "__main__":
    main()
```

```
Inserisci la domanda: Tell me about Medrysone

Contesto:
Medrysone 0.7888171672821045
Medazepam 0.7470438488377197
Medroxyprogesterone acetate 0.7346364259719849

Risposta dal modello:
Based on the provided context, Medrysone is a topical anti-inflammatory corticosteroid for ophthalmic use. It is used to treat various eye conditions such as allergic conjunctivitis, vernal conjunctivitis, episcleritis, and epinephrine sensitivity. The mechanism of action of Medrysone is not fully understood, but it is thought to act by inducing phospholipase A2 inhibitory proteins, which control the biosynthesis of potent mediators of inflammation such as prostaglandins and leukotrienes. Medrysone has been approved for use and is a solid substance.
```

Figura 4.2 Esempio funzionamento con embeddings

In questo modo, il modello, riesce a rispondere accuratamente a domande la cui risposta si trova all'interno del nodo, come ad esempio: *"Talk me about Medrysone"*, *"What are the causes of hepatitis B virus infection?"*, *"What are the symptoms of cystic fibrosis?"*. Per queste domande, la risposta è contenuta in uno o più degli attributi del nodo, ed il modello è in grado di identificare nel KG le 3 entità più significative, analizzandone le informazioni per generare poi la risposta ottimale. Tuttavia, quando si sottopongono al LLM, domande

riguardanti le relazioni tra le varie entità, specialmente se non esplicite, il modello incontra delle difficoltà nella fase di retrieval. Per questo motivo si prova ad utilizzare un approccio alternativo per l'estrazione delle informazioni rilevanti.

4.4 Text2Cypher

La tecnica **Text2Cypher** consiste nel trasformare, grazie ad un modello di NLP, delle frasi scritte in linguaggio naturale in **cypher**, permettendo di navigare facilmente su grafi anche complessi. I limiti di questo approccio riguardano la qualità del modello nel comprendere appieno la richiesta dell'utente e la capacità di riuscire a generare anche query complesse. Nel nostro caso quindi, si sfrutta la conoscenza della sintassi cypher di Llama3, per generare, dall'input dell'utente, una query che riesca ad estrarre le informazioni più accurate: nella fase di retrieval non si utilizza più la cosine similarity. Innanzitutto si concatenano in un'unica stringa sia la richiesta dell'utente sia la struttura del KG, questa viene inviata a Llama3 chiedendogli di generare esclusivamente la query cypher. Di seguito il codice.

```
def query_context_cypher(question):
    CYPHER_GENERATION_TEMPLATE = """Task:Generate Cypher statement to query a graph database.
    Instructions:
    Use only the provided relationship types and properties in the schema. Do not use any other
    ↳ relationship types or properties that are not provided.
    Database Schema:
        Node relationships:
            (:Node)-[:RELATION]->(:Node)

        Node properties:
            (:Node {node_id: INTEGER, node_index: INTEGER, node_name: STRING, node_source: STRING,
            ↳ node_type: STRING, atc_1: STRING, atc_2: STRING, atc_3: STRING, atc_4: STRING,
            ↳ category: STRING, description: STRING, group: STRING, indication: STRING,
            ↳ mechanism_of_action: STRING, pharmacodynamics: STRING, state: STRING, mondo_definition:
            ↳ STRING, orphanet_definition: STRING, umls_description: STRING, umls_description:
            ↳ STRING, mayo_complications: STRING, mayo_prevention: STRING, mayo_risk_factors: STRING,
            ↳ mayo_symptoms: STRING})

        Relationship Properties:
            [:RELATION {display_relation: STRING, relation: STRING }]

    Note: Do not include any explanations or apologies in your responses.
    Do not respond to any questions that might ask anything else than for you to construct a Cypher
    ↳ statement.
    Do not include any text except the generated Cypher statement.
    In the return clause, don't return the entire node, but only its attributes
    """ + f"""
    The question is:{question}"""
    resp = query_llama3_instruct(CYPHER_GENERATION_TEMPLATE)
    print("Query:")
    print(resp)
```

```

node=kg.query(resp)
print("context:")
if not node:
    print("Errore: Nessun nodo trovato.")
else:
    for a in node:
        print(a)
return node

```

In Fig. 4.3 è possibile vedere il funzionamento dell'approccio Text2cypher: la richiesta dell'utente, la query cypher generata, il contesto estratto e la risposta. In questo caso il modello ha generato una query errata, questa infatti non è riuscita a trovare nessun nodo, di conseguenza Llama3 ha generato una risposta basata solo sulla conoscenza appresa durante il suo addestramento e non con le informazioni presenti nel nostro KG, nullificando così l'idea alla base di un RAG.

```

Inserisci la domanda: Out of the given list, which is associated with "Tolazamide" and "Alclometasone". Given list is: ABO, HLA-DPA1, Empagliflozin, TNRC6B, JAK2, TERT

Query generata:
MATCH (n1:Node {node_name: "Tolazamide"})-[:RELATION]->(n2:Node)-[:RELATION]->(n3:Node)
WHERE n2.node_name IN ["Alclometasone"] AND n3.atc_1 IN ["ABO", "HLA-DPA1", "Empagliflozin", "TNRC6B", "JAK2", "TERT"]
RETURN n3.node_id, n3.node_index, n3.category

Contex:
Errore: Nessun nodo trovato.

Risposta dal modello cypher:
Based on the given list, Tolazamide and Alclometasone are both medications used to treat diabetes and skin conditions respectively. Therefore, I would associate Tolazamide with "JAK2" as both are related to diabetes treatment. Similarly, Alclometasone is a corticosteroid used for skin conditions, which makes me associate it with "TERT", another gene involved in skin cell development.

```

Figura 4.3 Esempio funzionamento Text2cypher

Analizzando la query prodotta (Fig. 4.3), possiamo notare come questa presenti delle discrepanze logiche. Per questo motivo, per aiutare il modello nella generazione della query, utilizziamo la tecnica **Few-shot**, concatenando alla stringa contenente la richiesta dell'utente e la struttura del database, alcuni esempi:

Examples: Here are a few examples of generated Cypher statements for particular questions:

```

# What's associated with "malaria" AND "duodenal ulcer"
MATCH (n)-->(m)-->(r)
WHERE n.node_name CONTAINS "malaria" AND r.node_name CONTAINS "duodenal ulcer"
RETURN m.node_name, m.category, m.pharmacodynamics,
       m.group, m.half_life, m.indication,
       m.mechanism_of_action, m.protein_binding,
       m.state, m.node_index, m.mayo_causes, m.mayo_complications, m.mayo_risk_factors,
       ↪ m.mayo_symptoms, m.mondo_definition
LIMIT 3

# Out of the given list, which Gene is associated with Takayasu arteritis and cancer. Given list
↪ is: SHTN1, HLA-B, SLC14A2, BTBD9, DTNB
MATCH (r)-->(n)-->(m)
WHERE n.node_name IN ["SHTN1", "HLA-B", "SLC14A2", "BTBD9", "DTNB"] AND m.node_name CONTAINS
       ↪ "Takayasu arteritis" AND r.node_name CONTAINS "cancer"
RETURN n.node_name, n.node_type
LIMIT 3

```

```
#What are the causes of cancer?
MATCH (n)
WHERE n.node_name CONTAINS "cancer"
RETURN n.node_name, n.category, n.pharmacodynamics,
       n.group, n.half_life, n.indication,
       n.mechanism_of_action, n.protein_binding,
       n.state, n.node_index, n.mayo_causes, n.mayo_complications, n.mayo_risk_factors,
       ↪ n.mayo_symptoms, n.mondo_definition
LIMIT 3
```

In Fig. 4.4 possiamo vedere come, a seguito della tecnica few-shots il modello sia riuscito a generare la query cypher corretta.

```
Inserisci la domanda: Out of the given list, which is associated with "Tolazamide" and "Alclometasone". Given list is: ABO, HLA-DPA1, Empagliflozin, TNRC6B, JAK2, TERT
Query:
MATCH (r)-->(n)-->(m)
WHERE n.node_name IN ["ABO", "HLA-DPA1", "Empagliflozin", "TNRC6B", "JAK2", "TERT"] and m.node_name CONTAINS "Tolazamide" and r.node_name CONTAINS "Alclometasone"
RETURN n.node_name, n.node_type LIMIT 3

Contesto:
{'n.node_name': 'Empagliflozin', 'n.node_type': 'drug'}

Risposta dal modello cypher:
Based on the given context, it appears that Tolazamide and Alclometasone are both drugs. Among the provided list of genes and proteins, only one is associated with these two drugs: Empagliflozin.
```

Figura 4.4 Esempio funzionamento Text2cypher con FewShot

4.5 Valutazione

Per valutare le prestazioni di un RAG, esistono diverse metriche [18]:

- Per la fase di **retrieval**:
 - **Relevance**, valuta quanto le informazioni estrapolate corrispondano con quelle espresse nella query, misura quindi quanto accuratamente il sistema riesca a trovare documenti che rispondano esattamente alla richiesta dell'utente;
 - **Accuracy**, valuta quanto siano accurati i documenti recuperati rispetto a un insieme di documenti candidati, misura quindi la capacità del sistema di ordinare correttamente i documenti in base alla loro rilevanza.
- Per la fase di **generation**:
 - **Relevance**, misura quanto la risposta sia pertinente all'argomento della query e soddisfi i requisiti della richiesta;
 - **Faithfulness**, valuta quanto la risposta generata sia fedele al contenuto dei documenti estratti;
 - **Correctness**, misura l'accuratezza della risposta generata confrontandola con una campione, detta **ground truth**.

The evaluation done by humans is still a very significant standard to compare the performance of generation models with one another or with the ground truth [18].

In questa sezione si procede a valutare il chatbot creato e di come i due approcci illustrati in precedenza (Embeddings e Text2Cypher) influenzino la qualità delle risposte, per farlo è stato utilizzato un dataset di domande mediche [14] appositamente creato per valutare un KG-RAG simile al nostro. A causa di alcune differenze, è stato possibile selezionare solo alcune delle domande, mentre per altre sono state necessarie delle modifiche.

Per valutare al meglio il comportamento del sistema, sono state selezionate domande aperte (*Tell me about Medrysone, What are the causes of hepatitis B virus infection?...*), a risposta multipla (*Out of the given list, which Gene is associated with membranous glomerulonephritis and uterine cervix. Given list is: PSCA, HLA-DQA1, KALRN, HLA-DRB5...*) e vero/falso (*Is cystic fibrosis a respiratory system disorder?...*).

Come già detto in precedenza la temperatura del LLM è stata impostata al minimo (0), limitando la "creatività" del modello in favore di risposte quasi deterministiche, questo è stato necessario poiché un valore maggiore avrebbe comportato un'incertezza delle query cypher generate, in quanto spesso si discostavano dagli esempi forniti. Per un corretto confronto tra i due approcci è stato utilizzato lo stesso valore in entrambi, perciò le risposte generate sono molto simili tra di loro quando il contesto aggiuntivo estratto è analogo.

Le risposte generate dai due approcci sono state sottoposte a degli esperti che hanno valutato, per ognuna, quale fosse la migliore e hanno attribuito alle metriche precedentemente descritte un voto da 1 a 5: i risultati ottenuti mostrano una relevance e faithfulness molto alta in entrambi gli approcci, ad indicare come il sistema riesca a generare delle risposte pertinenti e fedeli ai documenti estratti indipendentemente dal metodo utilizzato durante il retrieval. Text2Cypher ottiene un punteggio maggiore per quanto riguarda la correctness, a causa del fatto che gli embeddings delle volte non riescono ad estrapolare le informazioni corrette a domande riguardanti le relazioni tra entità (*What's associated with "hepatitis B" and "lymphoma"?*). Text2Cypher invece, riesce a rispondere perché per ogni tipologia di domanda selezionata dal dataset, è stato fornito al LLM un esempio di risoluzione.

4.6 Confronto Risultati

In questa sezione si analizzano i due approcci esposti precedentemente per la fase di retrieval, ovvero l'utilizzo degli Embeddings con la cosine similarity e Text2Cypher, esponendo i lati positivi e negati di ognuno.

La cosine similarity permette di individuare i nodi più rilevanti calcolando quanto i vettori di embeddings sono simili, questo però può diventare un'operazione lenta e computazionalmen-

te complessa, specialmente se il KG contiene un gran numero di nodi oppure se la dimensione dei vettori è elevata. Inoltre, nonostante la funzione di Neo4j `db.index.vector.queryNodes` (4.3), utilizzata per estrarre dal grafo le informazioni più rilevanti, dovrebbe restituire i nodi riordinati per lo score, questo delle volte non avviene: è stato quindi necessario restituire tutti i nodi e poi in seguito utilizzare la funzione `sort` presente in Python, con complessità $O(n \log n)$, per ordinarli correttamente.

La limitazione maggiore degli embeddings, come già esposto, riguarda il non riuscire a cogliere le relazioni tra le entità; un'altra è che tende a dare molta importanza allo score calcolato dalla cosine similarity, citandolo spesso nelle risposte, questo potrebbe rappresentare un problema in quanto per l'utente questo valore non ha alcun significato. Una possibile soluzione potrebbe essere il non fornire lo score, in questo modo però si priverebbe il modello di un parametro fondamentale, in quanto gli permette di avere un certo "grado di sicurezza" sulla risposta generata. In Text2Cypher tutto ciò non può avvenire in quanto non è presente alcun score, eliminando alla radice questo problema.

Text2Cypher sfrutta le potenzialità del LLM per generare dinamicamente la query cypher più adeguata per estrarre le informazioni pertinenti, questo procedimento permette di eseguire interrogazioni sul grafo complesse, dando maggior rilevanza alle relazioni esplicite tra i nodi ed inferendone altre implicite. Presenta anche vari svantaggi, tra i quali le difficoltà dovute a richieste ambigue e che presentano errori di sintassi: una semplice lettera sbagliata nel nome di un'entità oppure l'utilizzo di una lettera maiuscola o minuscola, può portare alla generazione di una query cypher errata in quanto la ricerca si basa su **entity matching**; questo non può avvenire con gli embeddings in quanto si valuta la somiglianza tra i nodi.

Un altro problema legato a Text2Cypher riguarda la quantità di informazioni estratte dal grafo, infatti con gli embeddings vengono selezionati soltanto i 3 nodi più rilevanti, mentre con Text2Cypher non è possibile prevederlo. La query cypher generata potrebbe infatti recuperare troppe informazioni, andando così ad esaurire i token disponibili nella finestra di dialogo del LLM.

Il metodo Text2Cypher ha dimostrato i risultati desiderati nell'estrazione delle informazioni, solo dopo aver fornito in input degli esempi di query simili alla richiesta, senza di questi le query generate erano spesso, seppur con la sintassi giusta, prive di senso logico. Questo rappresenta la più grande limitazione di questo approccio: per la creazione di un chatbot basato interamente su Text2Cypher, bisognerebbe fornire al modello un numero estremamente alto di esempi che siano in grado di coprire tutte le possibili richieste dell'utente. Delle possibili soluzioni potrebbero essere l'utilizzo di un LLM differente, possibilmente addestrato con più parametri, questo infatti potrebbe aver una maggior comprensione del testo e delle intenzioni dell'utente e una maggior consapevolezza nella traduzione in cypher; oppure l'utilizzo di un

architettura ibrida che sfrutta sia i punti di forza degli embeddings sia di Text2Cypher. Pur avendo ciascuno i propri punti di forza e debolezza, entrambi gli approcci hanno mostrato ottimi risultati dal punto di vista della riduzione hallucinations, in Fig. 4.5 e 4.6 possiamo vedere la differenza nelle risposte generate da Llama3, rispettivamente senza e con l'utilizzo dei RAG: grazie all'utilizzo dei RAG, il modello è riuscito a generare la risposta corretta, anche se non è in grado di motivarla in quanto questa non è presente nel contesto fornitogli.

```
>>> Out of the given list, which Gene is associated with membranous glomerulonephritis and uterine cervix. Given list is
... : PSCA, HLA-DQA1, KALRN, HLA-DRB5
According to various sources, the gene associated with membranous glomerulonephritis is HLA-DRB5.

Membranous glomerulonephritis (MGN) is a type of glomerulonephritis that affects the kidneys. It's caused by an
immune response to antigens in the basement membrane of glomerular capillaries, leading to deposition of immune
complexes and subsequent inflammation.

As for uterine cervix, HLA-DRB5 has also been implicated in cervical cancer susceptibility. Specifically, a study
found that genetic variants in HLA-DRB5 were associated with an increased risk of developing invasive cervical
cancer.

So, the answer is HLA-DRB5!
```

Figura 4.5 Esempio funzionamento senza RAG

```
Inserisci la domanda: Out of the given list, which Gene is associated with membranous glomerulonephritis and uterine cervix. Given list is: PSCA, HLA-DQA1, KALRN, HLA-DRB5
Risposta:
Based on the given context and list of genes, I can infer that HLA-DQA1 is associated with membranous glomerulonephritis and uterine cervix.
```

Figura 4.6 Esempio funzionamento con RAG

5

Conclusioni

Gli LLM sono ad oggi, un incredibile strumento versatile con un'enormità di pregi ma anche di difetti, tra questi troviamo senz'alcun ombra di dubbio i problemi etici, la presenza di bias e le hallucinations. Per ridurre quest'ultime, sono state presentate varie promettenti tecniche, tra cui i RAG, una tecnica con un enorme potenziale che ha come punto di forza la possibilità di infondere nuove informazioni in un modello senza dover sostenere i costi di un classico ri-addestramento.

In questo elaborato sono stati discussi vari approcci per la costruzione di un RAG, ponendo maggiore attenzione in particolare sull'implementazione di un grafo di conoscenza per rappresentare le informazioni; sono stati riportati inoltre vari esempi pratici, nei quali si evidenziava un notevole miglioramento nella riduzione delle hallucinations a discapito, delle volte, delle prestazioni generali durante la fase di retrieval (Vedi Tab. 3.1).

Nonostante le varie limitazioni che questi ancora presentano, i RAG si candidano ad essere in futuro un'ottima soluzione per limitare le hallucinations, attenuando così uno dei maggiori problemi legati agli LLM.

6

Codice

6.1 Calcolo Embeddings

```
from dotenv import load_dotenv
import os
from langchain_community.graphs import Neo4jGraph
from sentence_transformers import SentenceTransformer
import numpy as np

# Warning control
import warnings
warnings.filterwarnings("ignore")

# Load environment variables
load_dotenv('.env', override=True)
NEO4J_URI = os.getenv('NEO4J_URI')
NEO4J_USERNAME = os.getenv('NEO4J_USERNAME')
NEO4J_PASSWORD = os.getenv('NEO4J_PASSWORD')
NEO4J_DATABASE = os.getenv('NEO4J_DATABASE')

# Connect to the knowledge graph instance using LangChain
kg = Neo4jGraph(
    url=NEO4J_URI, username=NEO4J_USERNAME, password=NEO4J_PASSWORD, database=NEO4J_DATABASE
)

# Load SentenceTransformer model
model = SentenceTransformer('all-MiniLM-L6-v2')

def nodes_and_attributes():
    # Query 1: Recupera nodi di tipo "drug"
    nodes_drug = kg.query("""
```

```

MATCH (n {node_type:"drug"})
RETURN n.node_name AS name, n.category AS category, n.pharmacodynamics AS pharmacodynamics,
       n.group AS group, n.half_life AS half_life, n.indication AS indication,
       n.mechanism_of_action AS mechanism_of_action, n.protein_binding AS protein_binding,
       n.state AS state, n.node_index as node_index
"""
# Query 2: Recupera nodi di tipo "disease"
nodes_disease = kg.query("""
MATCH (n) WHERE n.node_index IN [30388, 28714, 94687, 37752, 94707]
RETURN n.node_name AS node_name,
       n.mayo_causes AS mayo_causes,
       n.mayo_complications AS mayo_complications,
       n.mayo_risk_factors AS mayo_risk_factors,
       n.mayo_symptoms AS mayo_symptoms,
       n.mondo_definition AS mondo_definition,
       n.node_type AS node_type,
       n.umls_description AS umls_description,
       n.node_index AS node_index
""")
# Query 3: Recupera nodi senza embedding
nodes_missing_embeddings = kg.query("""
MATCH (n) WHERE n.embedding IS NULL
RETURN n.node_index AS node_index, n.node_name AS node_name,
       n.node_source AS node_source, n.node_type AS node_type
""")
# Concatena i risultati delle tre query
all_nodes = nodes_drug + nodes_disease + nodes_missing_embeddings
return all_nodes

def calculate_embeddings():
    nodes = nodes_and_attributes()
    embeddings = {}
    for node in nodes:
        node_index = node['node_index']
        # Concatenazione attributi
        text_to_embed = ' '.join(filter(None, [node[attr] for attr in node.keys() if attr !=
        ↪ 'node_index']))
        # Calcola l'embedding per il testo concatenato
        if text_to_embed:
            embeddings[node_index] = model.encode(text_to_embed)
        else:
            embeddings[node_index] = np.zeros(model.get_sentence_embedding_dimension())
    return embeddings

def update_nodes_with_embeddings(embeddings):
    for node_index, embedding in embeddings.items():
        embedding_list = embedding.tolist()
        update_query = f"""
MATCH (n {{node_index: '{node_index}'}})
SET n.embedding = {embedding_list}
"""
        kg.query(update_query)

if __name__ == "__main__":

```

```

node_embeddings = calculate_embeddings()
update_nodes_with_embeddings(node_embeddings)
print("Aggiornamento degli embeddings completato.")

```

6.2 Pipeline

```

import subprocess
import os
from langchain_community.graphs import Neo4jGraph
from dotenv import load_dotenv
from sentence_transformers import SentenceTransformer

# Warning control
import warnings
warnings.filterwarnings("ignore")

# Load environment variables
load_dotenv('.env', override=True)
NEO4J_URI = os.getenv('NEO4J_URI')
NEO4J_USERNAME = os.getenv('NEO4J_USERNAME')
NEO4J_PASSWORD = os.getenv('NEO4J_PASSWORD')
NEO4J_DATABASE = os.getenv('NEO4J_DATABASE')

# Connect to the knowledge graph using LangChain
kg = Neo4jGraph(
    url=NEO4J_URI, username=NEO4J_USERNAME, password=NEO4J_PASSWORD, database=NEO4J_DATABASE
)

# Load SentenceTransformer model
model = SentenceTransformer('all-MiniLM-L6-v2')

def query_llama3_instruct(input_text):
    # Command to run the llama3:instruct model using ollama
    command = ['ollama', 'run', 'CustomLlama3:latest']
    # Start subprocess
    process = subprocess.Popen(command, stdin=subprocess.PIPE, stdout=subprocess.PIPE,
        ↪ stderr=subprocess.PIPE, text=True, encoding='utf-8')
    stdout, stderr = process.communicate(input=input_text)
    # Check success
    if process.returncode == 0:
        return stdout
    else:
        return f"Error: {stderr}"

def calculate_embedding(text):
    embedding = model.encode(text)
    return embedding

def query_context(question_embedding):
    # Query to find context based on question embedding
    nodes = kg.query("""
CALL db.index.vector.queryNodes(
    'fullindex',

```

```

129375,
$question_embedding
) YIELD node AS n, score
RETURN n.node_name AS name, n.category AS category, n.pharmacodynamics AS pharmacodynamics,
       n.group AS group, n.half_life AS half_life, n.indication AS indication,
       n.mechanism_of_action AS mechanism_of_action, n.protein_binding AS protein_binding,
       n.state AS state, n.node_index AS node_index, n.node_type AS Type, n.node_source AS Source,
       n.mayo_causes AS mayo_causes,
       n.mayo_complications AS mayo_complications,
       n.mayo_risk_factors AS mayo_risk_factors,
       n.mayo_symptoms AS mayo_symptoms,
       n.mondo_definition AS mondo_definition, score
""" , params={"question_embedding": question_embedding})
return nodes

def query_context_cypher(question):
    CYPHER_GENERATION_TEMPLATE = """Task:Generate Cypher statement to query a graph database.
Instructions:
Use only the provided relationship types and properties in the schema. Do not use any other
↪ relationship types or properties that are not provided.
Database Schema:
Node relationships:
    (:Node)-[:RELATION]->(:Node)

Node properties:
    (:Node {node_id: INTEGER, node_index: INTEGER, node_name: STRING, node_source: STRING,
    ↪ node_type: STRING, atc_1: STRING, atc_2: STRING, atc_3: STRING, atc_4: STRING,
    ↪ category: STRING, description: STRING, group: STRING, indication: STRING,
    ↪ mechanism_of_action: STRING, pharmacodynamics: STRING, state: STRING,mondo_definition:
    ↪ STRING, orphanet_definition: STRING, umls_description: STRING, umls_description:
    ↪ STRING, mayo_complications: STRING, mayo_prevention: STRING, mayo_risk_factors: STRING,
    ↪ mayo_symptoms: STRING})

Relationship Properties:
    [:RELATION {display_relation: STRING, relation: STRING }]

Note: Do not include any explanations or apologies in your responses.
Do not respond to any questions that might ask anything else than for you to construct a Cypher
↪ statement.
Do not include any text except the generated Cypher statement.
Use the examples for respond to similiary question.

Examples: Here are a few examples of generated Cypher statements for particular questions:
# What's associated with "malaria" AND "duodenal ulcer"
MATCH (n)-->(m)-->(r)
WHERE n.node_name contains "malaria" AND r.node_name contains"duodenal ulcer"
RETURN m.node_name, m.category, m.pharmacodynamics,
       m.group, m.half_life, m.indication,
       m.mechanism_of_action, m.protein_binding,
       m.state, m.node_index, m.mayo_causes, m.mayo_complications, m.mayo_risk_factors,
       ↪ m.mayo_symptoms, m.mondo_definition limit 3

# Out of the given list, which Gene is associated with Takayasu arteritis and cancer. Given list
↪ is: SHTN1, HLA-B, SLC14A2, BTBD9, DTNB
MATCH (r)-->(n)-->(m)

```

```

WHERE n.node_name IN ["SHTN1", "HLA-B", "SLC14A2", "BTBD9", "DTNB"] and m.node_name Contains
↪ "Takayasu arteritis" and r.node_name contains "cancer"
RETURN n.node_name, n.node_type LIMIT 3

#What are the causes of cancer?
MATCH (n)
WHERE n.node_name CONTAINS "cancer"
RETURN n.node_name, n.category, n.pharmacodynamics,
       n.group, n.half_life, n.indication,
       n.mechanism_of_action, n.protein_binding,
       n.state, n.node_index, n.mayo-causes, n.mayo-complications, n.mayo-risk-factors,
       ↪ n.mayo-symptoms, n.mondo-definition
LIMIT 3

#What are the symptoms of cancer?
MATCH (n)
WHERE n.node_name CONTAINS "cancer"
RETURN n.node_name, n.category, n.pharmacodynamics,
       n.group, n.half_life, n.indication,
       n.mechanism_of_action, n.protein_binding,
       n.state, n.node_index, n.mayo-causes, n.mayo-complications, n.mayo-risk-factors,
       ↪ n.mayo-symptoms, n.mondo-definition
LIMIT 3

# Tell me about Copper
MATCH (n)
WHERE n.node_name CONTAINS "Copper"
RETURN n.node_name, n.category, n.pharmacodynamics,
       n.group, n.half_life, n.indication,
       n.mechanism_of_action, n.protein_binding,
       n.state, n.node_index, n.mayo-causes, n.mayo-complications, n.mayo-risk-factors,
       ↪ n.mayo-symptoms, n.mondo-definition
LIMIT 3
""" +f"""
The question is:
{question}"""
resp = query_llama3_instruct(CYPHER_GENERATION_TEMPLATE)
print("Query:")
print(resp)
node=kg.query(resp)
print("context:")
if not node:
    print("Errore: Nessun nodo trovato.")
else:
    for a in node:
        print(a)
return node

def main():
    question = input("Inserisci la domanda: ")
    # Calculate the embedding
    question_embedding = calculate_embedding(question)
    context=query_context(question_embedding)
    context.sort(key=lambda obj: obj['score'], reverse=True)

```

```

print("\nContesto:")
for node in context[:3]:
    print(node['name'], node['score'])

top_context = context[:3]
# Concatenate context and question
input_text = f"""TASK: You are an AI assistant. Respond in paragraph using only the context
↪ provided, giving more relevance to the one with the higher score.
QUESTION: {question}
CONTEXT: {top_context} """

# Get the response from the model
response = query_llama3_instruct(input_text)
print("\nRisposta dal modello:")
print(response)

# Use llama for the query
node=query_context_cypher(question)
# Concatenate context and question
input_text = f"""TASK: You are an AI assistant. Respond in paragraph using only the context
↪ provided.
QUESTION: {question}
CONTEXT: {node} """

resp = query_llama3_instruct(input_text)
print("Risposta dal modello cypher:")
print(resp)

if __name__ == "__main__":
    main()

```

Bibliografia

- [1] Aziz, I. (2024). Dynamic few-shot prompting to create captivating digital content.
- [2] Chandak, P., Huang, K., and Zitnik, M. (2022). Building a knowledge graph to enable precision medicine. *bioRxiv*.
- [3] Developers, G. (2023). Incorporamenti: traduzione in uno spazio di dimensioni inferiori.
- [4] Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., Wang, M., and Wang, H. (2024). Retrieval-augmented generation for large language models: A survey.
- [5] Heston, T. F. and Khun, C. (2023). Prompt engineering in medical education. *International Medical Education*, 2(3):198–205.
- [6] Huang, L., Yu, W., Ma, W., Zhong, W., Feng, Z., Wang, H., Chen, Q., Peng, W., Feng, X., Qin, B., and Liu, T. (2023). A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions.
- [7] Kojima, T., Gu, S. S., Reid, M., Matsuo, Y., and Iwasawa, Y. (2023). Large language models are zero-shot reasoners.
- [8] Kong, S. C. L. B. Z. M. M. G. A. G. S. S. O. P. R. (2023). Examples of prompts that cause chatgpt-4 to hallucinate.
- [9] Li, D.-J., Kao, Y.-C., Tsai, S.-J., Bai, Y.-M., Yeh, T.-C., Chu, C.-S., Hsu, C.-W., Cheng, S.-W., Hsu, T.-W., Liang, C.-S., and Su, K.-P. (2024). Comparing the performance of chatgpt gpt-4, bard, and llama-2 in the taiwan psychiatric licensing examination and in differential diagnosis with multi-center psychiatrists. *Psychiatry and Clinical Neurosciences*, 78(6):347–352.
- [10] Monigatti, L. (2023). Retrieval-augmented generation (rag): From theory to langchain implementation.
- [11] Perković, G., Drobnjak, A., and Botički, I. (2024). Hallucinations in llms: Understanding and addressing challenges. In *2024 47th MIPRO ICT and Electronics Convention (MIPRO)*, pages 2084–2088.
- [12] Sanmartin, D. (2024). Kg-rag: Bridging the gap between knowledge and creativity.
- [13] Shaikh, B. (2023). Understanding distance metrics in vector embeddings: Cosine similarity, euclidean distance, and dot product.

- [14] Soman, K., Rose, P. W., Morris, J. H., Akbas, R. E., Smith, B., Peetoom, B., Villouta-Reyes, C., Ceronio, G., Shi, Y., Rizk-Jackson, A., et al. (2023). Biomedical knowledge graph-enhanced prompt generation for large language models. *arXiv preprint arXiv:2311.17330*.
- [15] Šekrst, K. (forthcoming). Unjustified untrue "beliefs": Ai hallucinations and justification logics. In Świętorzecka, K., Grgić, F., and Brozek, A., editors, *Logic, Knowledge, and Tradition. Essays in Honor of Srećko Kovac*.
- [16] Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q., and Zhou, D. (2023). Chain-of-thought prompting elicits reasoning in large language models.
- [17] Xu, Z., Cruz, M. J., Guevara, M., Wang, T., Deshpande, M., Wang, X., and Li, Z. (2024). Retrieval-augmented generation with knowledge graphs for customer service question answering. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR 2024*. ACM.
- [18] Yu, H., Gan, A., Zhang, K., Tong, S., Liu, Q., and Liu, Z. (2024). Evaluation of retrieval-augmented generation: A survey.
- [19] Zhang, Y., Li, Y., Cui, L., Cai, D., Liu, L., Fu, T., Huang, X., Zhao, E., Zhang, Y., Chen, Y., Wang, L., Luu, A. T., Bi, W., Shi, F., and Shi, S. (2023). Siren's song in the ai ocean: A survey on hallucination in large language models.